

Object Relational Mapping (ORM)

¿Qué es un ORM?

ORM (Object-Relational Mapping), o Mapeo Objeto-Relacional, es una técnica que permite establecer una correspondencia entre los objetos de un programa orientado a objetos y las estructuras de una base de datos relacional, como tablas, columnas y relaciones.

De esta manera, el programador puede trabajar principalmente con objetos del lenguaje de programación y el ORM se encarga de realizar las operaciones necesarias sobre la base de datos.

El ORM se encarga de convertir el objeto de una clase en tablas donde los atributos son los campos de la tabla y cada fila será un objeto.

¿Cuáles son los ORM más conocidos del mercado?

Existen numerosos ORM y su elección depende principalmente del lenguaje. Entre los mas destacados se encuentran:

- **SQL ALCHEMY** para **PYTHON**
- **GORM** para **GO**
- **TypeORM** para **JavaScript**
- **HIBERNATE** para **JAVA**
- **Entity Framework** para **C#**
- **Eloquent** para **PHP**
- **Active Record** para **Ruby**
- **Peewee** para **PYTHON**
- **Sequalize** para **JavaScript**

¿Qué lenguajes de programación pueden interactuar con ORM?

Los ORM no pertenecen a un único lenguaje de programación. Existen implementaciones para numerosos lenguajes que permiten trabajar con bases de datos relacionales utilizando las estructuras propias de cada lenguaje.

Algunos ejemplos podrían ser Java, GO, Python, Ruby, JavaScript, PHP, C#.

¿Cómo funciona un ORM?

Podemos tener una clase llamada **Persona**:

Persona
-DNI : Int
-Nombre: String
-Apellido: String

El **ORM** va a asociarla con una tabla de la siguiente forma:

TABLA PERSONA		
DNI	Nombre	Apellido

Clase	Base de datos
-------	---------------

Clase Persona → Tabla PERSONA

atributo id → columna id

atributo nombre → columna nombre

objeto Persona → fila de PERSONA

Cuando el programa necesita consultar, insertar, modificar o eliminar información, el ORM transforma las operaciones realizadas sobre los objetos en las operaciones correspondientes sobre la base de datos.

Por ejemplo:

programa:

```
persona = nueva Persona()
```

```
persona.nombre = "Juan"
```

```
persona.apellido = "Pérez"
```

```
persona.guardar()
```

El ORM puede traducir esa operación a una sentencia SQL equivalente a:

INSERT INTO PERSONA (nombre, apellido)

VALUES ('Juan', 'Pérez');

En Hibernate, por ejemplo, existen operaciones para buscar, persistir, modificar y eliminar entidades, además de mecanismos para representar asociaciones, herencia y otros elementos del modelo orientado a objetos.

¿Cuáles son sus ventajas y desventajas?

Como ventajas podemos encontrar:

- **Menos código repetido**

Sin un ORM, una aplicación puede necesitar escribir manualmente muchas consultas SQL y código para convertir los resultados de esas consultas en objetos.

El ORM automatiza gran parte de esta tarea.

- **Manejo de relaciones**

Los ORM permiten representar relaciones entre objetos y asociarlas con relaciones de la base de datos.

Por ejemplo:

Cliente 1 ————— * Venta

puede representarse mediante relaciones entre clases y, en la base de datos, mediante una clave foránea.

- **Portabilidad**

Muchos ORM permiten trabajar con diferentes motores de bases de datos sin tener que modificar completamente el código de la aplicación.

Por otro lado, podríamos encontrar las siguientes desventajas:

- **Pérdida de control sobre SQL**

Al abstraer las consultas SQL, el programador puede perder cierto control sobre las consultas que finalmente se ejecutan en la base de datos.

- **Posible pérdida de rendimiento**

Una consulta generada automáticamente por un ORM no siempre será la consulta SQL más eficiente posible.

Por eso, aunque los ORM facilitan el desarrollo, **no eliminan la necesidad de conocer SQL y el funcionamiento de las bases de datos.**

- **Curva de aprendizaje**

Aunque inicialmente puede parecer más sencillo que escribir SQL, cada ORM tiene sus propias reglas, configuraciones, métodos y mecanismos de mapeo.

Por ejemplo, Hibernate posee conceptos como Session, EntityManager, asociaciones, estrategias de carga y lenguaje de consultas propio.

Realizar de forma gráfica su funcionamiento.

El flujo general de un ORM se puede representar de la siguiente forma:

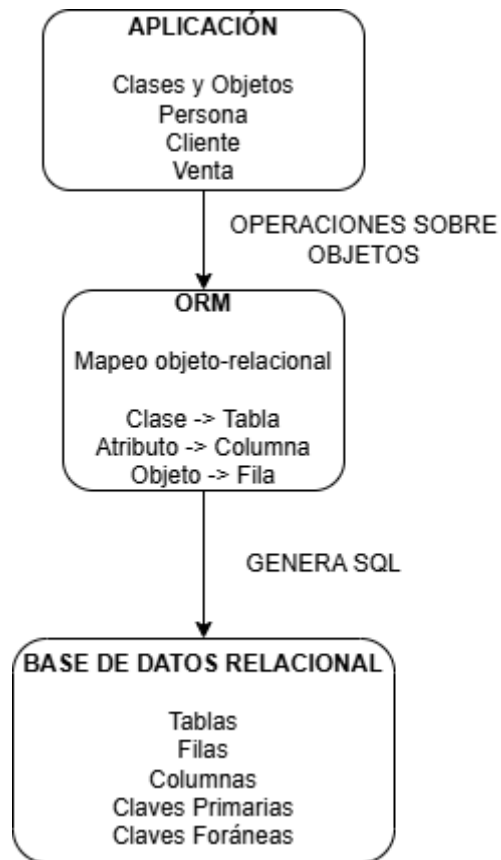


DIAGRAMA HECHO EN: <https://app.diagrams.net/>

Si consultamos información, veríamos el flujo de forma inversa:

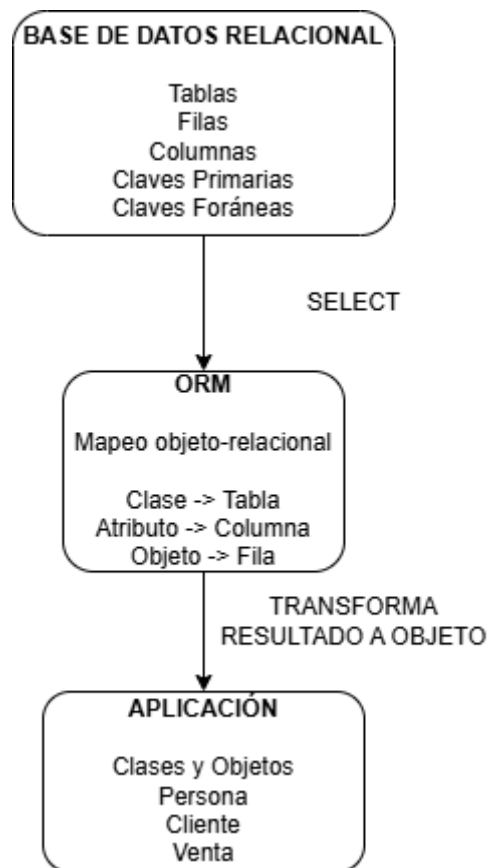


DIAGRAMA HECHO EN: <https://app.diagrams.net/>

Por lo tanto, el ORM actúa como una capa de abstracción entre la aplicación y la base de datos.